

→ Straight forward option → binary I/O and compression

C++ is a faster choice

Required :

Structure

→ At least 3 separate executables : parser , merger and query processor

Each one writes its output to disk files for the next one to read

Data :

→ Index the MS MARCO passage dataset

→ Assume the data doesn't fit in memory.

Build the index by sorting in chunks , writing temp files and then merging them

→ Don't crash on non-English or weird characters

Final index:

→ only 3 to 5 files

→ Include

- Inverted lists (word → list of doc IDs and frequencies)
- Lexicon (word → where its list starts in the file)
- page table (doc ID → Passage ID)
- Metadata such as the last doc ID in each block and block sizes

→ Lists must be blocked , binary and compressed

→ store doc IDs and frequencies in separate blocks , not interleaved.

→ Each posting holds a docID plus a frequency (or an impact score) and no word positions

Query processor

→ At startup , load the lexicon and page table into memory but never all the inverted lists

→ For each query :

- Look up the words in the lexicon
- Seek to each lists position on disk
- traverse the lists DART (document at a time)
- score with BM25
- return top 10

→ Support both AND (conjunctive) and OR (disjunctive) modes , chosen per query

→ After each query , wait for the next one in a loop

Inverted API List

→ Implement

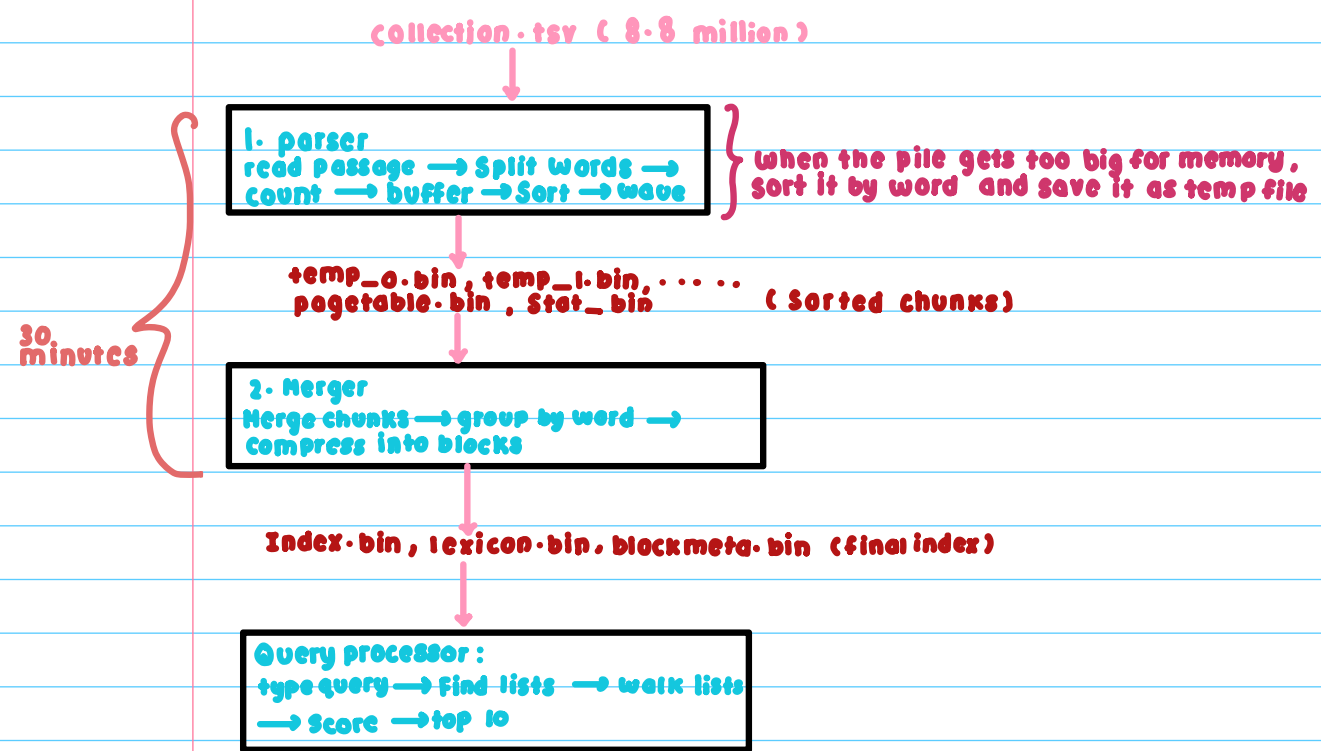
- openList ()
- LoadBlock ()

- closeList ()
- nextGEQ () → Forward seek
- getFreq () → Must write ourselves

→ Decompress lazily

→ I can use open source to compress a single block

Pipeline



- parser : passages → Sorted
- Merger : Sorted → one compressed index
- Query processor → top 10 results

```

$ ./query
Loading lexicon ....
Loading page table ....
Loading stats . .
  
```

```

Mode [ and / or ] =
Query :

Top 10 results ( __ mode , __ matches , speed ) :
#   Passage ID   score
1
2
3
...
10
  
```

- Read mode
 - Read query
 - tokenise (same rules as the parser)
 - openList () for each word
 - AND or OR DAAT Loop
 - bm25 () per match
 - DocID → passage ID via page table → print closeList ()
- Repeat